15 — Prompt Engineering & LLM Patterns

Time: ~4-5 hours | **Level**: Advanced

What you'll learn:

- Prompt engineering: zero-shot, few-shot, system prompts
- Advanced prompting: chain-of-thought, self-consistency, tree-of-thought
- Structured output: JSON mode, constrained generation, Pydantic parsing
- LLM-as-Judge: using one model to evaluate another
- Function calling and tool use patterns
- Agent architectures: ReAct, plan-and-execute
- Cost and latency optimization strategies

Prerequisites: Notebooks 07-10, 14 (Transformers, HuggingFace, fine-tuning, embeddings)

Prompt Engineering Is Software Engineering

A well-engineered prompt is the difference between a model that hallucinates and one that produces reliable, structured output. This notebook teaches you systematic prompting — not ad-hoc "prompt hacking".

In [ ]:
```
import json
import re
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from dataclasses import dataclass, field
from typing import List, Optional, Dict, Any

sns.set_theme(style='whitegrid', font_scale=1.1)
np.random.seed(42)
```

1. Prompt Engineering Foundations

All modern LLMs use a **chat format** with three roles:

Role	Purpose	Example
System	Sets behaviour, personality, constraints	"You are a clinical mental health assistant"
User	The question or instruction	"Assess this patient's risk level"

Role	Purpose	Example
Assistant	The model's response	"Based on the scores provided..."

Zero-shot vs Few-shot

- **Zero-shot**: Just give the instruction, no examples
- **One-shot**: Give one example, then the task
- **Few-shot**: Give 2-5 examples, then the task

Few-shot prompting dramatically improves output quality and format consistency.

In [ ]:
```python
# ── Prompt templates for mental health assessment ──────────────

# Zero-shot prompt
zero_shot = {
    'system': 'You are a clinical mental health assessment assistant.',
    'user': (
        'Assess the following patient:\n'
        'Age: 34, Gender: Female\n'
        'Depression Score: 0.72, Anxiety Score: 0.65\n'
        'Symptoms: persistent low mood, insomnia, loss of appetite\n'
        'Therapy History: None\n\n'
        'Provide a severity rating and clinical recommendation.'
    )
}

# Few-shot prompt (with examples)
few_shot = {
    'system': (
        'You are a clinical mental health assessment assistant. '
        'For each patient, provide:\n'
        '1. Severity: High/Moderate/Low\n'
        '2. Key Concerns: bullet list\n'
        '3. Recommendation: specific next steps'
    ),
    'examples': [
        {
            'user': 'Patient: Age 28, Male. Depression: 0.35, Anxiety: 0.20.
                    'Symptoms: occasional worry about work.',
            'assistant': (
                'Severity: Low\n'
                'Key Concerns:\n'
                '- Mild workplace anxiety within normal range\n'
                '- No significant depressive symptoms\n'
                'Recommendation: Monitor. Suggest stress management techniqu
                'Follow up in 3 months.'
            )
        },
        {
            'user': 'Patient: Age 52, Female. Depression: 0.85, Anxiety: 0.7
                    'Symptoms: suicidal ideation, severe insomnia, social wi
            'assistant': (
                'Severity: High\n'
```

```
                'Key Concerns:\n'
                '- Active suicidal ideation requires immediate risk assessme
                '- Severe depression and anxiety co-occurrence\n'
                '- Social withdrawal indicates deteriorating support network
                'Recommendation: URGENT. Conduct C-SSRS risk assessment. '
                'Initiate crisis intervention protocol. Consider SSRI + CBT
                'Schedule weekly follow-ups.'
            )
        }
    ],
    'user': (
        'Patient: Age 34, Female. Depression: 0.72, Anxiety: 0.65. '
        'Symptoms: persistent low mood, insomnia, loss of appetite. '
        'Therapy History: None.'
    )
}

print('=== Zero-Shot Prompt ===')
print(f'System: {zero_shot["system"]}')
print(f'User: {zero_shot["user"][:100]}...')
print(f'\n=== Few-Shot Prompt (2 examples) ===')
print(f'System: {few_shot["system"][:80]}...')
print(f'Examples: {len(few_shot["examples"])} provided')
print(f'User: {few_shot["user"][:80]}...')
print(f'\n💡 Few-shot prompts teach the model both FORMAT and REASONING by e
print(f'   The model will follow the same Severity/Concerns/Recommendation s
```

2. Advanced Prompting Techniques

Chain-of-Thought (CoT)

Instead of asking for a direct answer, ask the model to **think step by step**.

This dramatically improves reasoning accuracy, especially for:

- Multi-step problems
- Risk assessment (weighing multiple factors)
- Diagnostic reasoning

Self-Consistency

Run the same CoT prompt N times → take the **majority answer**. Reduces variance from single-sample generation.

Tree-of-Thought (ToT)

Explore multiple reasoning branches, evaluate each, and select the best path.

Technique	Accuracy Gain	Cost	Best For
Zero-shot	Baseline	1x	Simple tasks

Technique	Accuracy Gain	Cost	Best For
Few-shot	+10-30%	1.5x	Format consistency
CoT	+20-40%	2x	Reasoning tasks
Self-consistency	+5-15% over CoT	Nx	High-stakes decisions
ToT	+10-20% over CoT	5-10x	Complex multi-step reasoning

In [ ]:
```python
# ── Chain-of-Thought prompting ──────────────────────────────

standard_prompt = (
    'Patient: Age 45, Male. Depression: 0.68, Anxiety: 0.55. '
    'History: previous suicide attempt 2 years ago, currently on sertraline
    'reports medication non-compliance for past 2 weeks.\n\n'
    'What is the risk level? Answer: High, Moderate, or Low.'
)

cot_prompt = (
    'Patient: Age 45, Male. Depression: 0.68, Anxiety: 0.55. '
    'History: previous suicide attempt 2 years ago, currently on sertraline
    'reports medication non-compliance for past 2 weeks.\n\n'
    'Assess the risk level step by step:\n'
    '1. First, identify all risk factors present\n'
    '2. Then, identify any protective factors\n'
    '3. Weigh risk vs protective factors\n'
    '4. Determine the risk level: High, Moderate, or Low\n\n'
    'Think through each step carefully:'
)

# Simulated CoT response (what a model would generate)
cot_response = """Step 1 - Risk Factors:
- Previous suicide attempt (strongest predictor of future attempts)
- Medication non-compliance for 2 weeks (risk of withdrawal + symptom relaps
- Elevated depression score (0.68 = moderately severe)
- Co-occurring anxiety (0.55 = moderate)
- Male gender (higher completed suicide rate)

Step 2 - Protective Factors:
- Currently on medication (indicates engagement with treatment)
- No current suicidal ideation mentioned (but must be assessed)

Step 3 - Weighing:
- Multiple high-risk factors, especially previous attempt + non-compliance
- Protective factors are limited
- Medication discontinuation without medical supervision is dangerous

Step 4 - Risk Level: HIGH
Previous suicide attempt combined with recent medication non-compliance
creates an acute risk escalation requiring immediate clinical attention."""

print('=== Standard Prompt ===')
print(standard_prompt)
print('\nExpected model output: "Moderate" or "High" (no reasoning visible)'
```

```
print('\n=== Chain-of-Thought Prompt ===')
print(cot_prompt)
print('\nExpected model output (with reasoning):')
print(cot_response)

print('\n💡 CoT forces the model to SHOW its reasoning, which:')
print('   1. Improves accuracy (thinking step-by-step catches errors)')
print('   2. Enables auditing (clinicians can verify the reasoning)')
print('   3. Builds trust (transparent decision-making)')
```

In [ ]:

```
# ── Self-consistency: majority vote over multiple CoT runs ──────

def self_consistency(responses, extract_fn):
    """Take majority vote over multiple model responses."""
    answers = [extract_fn(r) for r in responses]
    from collections import Counter
    vote_counts = Counter(answers)
    majority = vote_counts.most_common(1)[0]
    return {
        'final_answer': majority[0],
        'confidence': majority[1] / len(responses),
        'all_votes': dict(vote_counts)
    }

# Simulated 5 CoT runs (in practice, you'd call the LLM 5 times with tempera
simulated_responses = [
    '...Risk Level: HIGH. Previous suicide attempt is the dominant factor.',
    '...Risk Level: HIGH. Medication non-compliance creates acute danger.',
    '...Risk Level: MODERATE. Scores are moderately elevated but no current
    '...Risk Level: HIGH. Combination of history and non-compliance is criti
    '...Risk Level: HIGH. Must assess for current suicidal ideation immediat
]

def extract_risk(text):
    """Extract risk level from model output."""
    match = re.search(r'Risk Level:\s*(HIGH|MODERATE|LOW)', text, re.IGNOREC
    return match.group(1).upper() if match else 'UNKNOWN'

result = self_consistency(simulated_responses, extract_risk)
print(f'Self-Consistency Result (5 runs):')
print(f'  Final answer: {result["final_answer"]}')
print(f'  Confidence: {result["confidence"]:.0%}')
print(f'  Vote distribution: {result["all_votes"]}')
print(f'\n💡 4 out of 5 runs said HIGH → high confidence in the assessment.'
print(f'   Self-consistency costs 5x but catches single-run errors.')
```

3. Structured Output

In production, you need **parseable output** — not free-form text.

Strategies:

1. **JSON mode**: Ask the model to output JSON

2. **Pydantic validation**: Parse + validate the output
3. **Constrained generation**: Force the model to follow a grammar/schema

This is critical for mental health systems where downstream code processes the model's output.

In [ ]:
```python
# ── Pydantic models for structured clinical output ──────────────
from pydantic import BaseModel, Field, validator
from enum import Enum

class Severity(str, Enum):
    HIGH = 'High'
    MODERATE = 'Moderate'
    LOW = 'Low'

class ClinicalAssessment(BaseModel):
    """Structured output schema for mental health assessment."""
    severity: Severity = Field(description='Overall risk severity level')
    confidence: float = Field(ge=0.0, le=1.0, description='Model confidence
    risk_factors: List[str] = Field(description='Identified risk factors')
    protective_factors: List[str] = Field(description='Identified protective
    recommended_actions: List[str] = Field(description='Clinical recommendat
    follow_up_days: int = Field(ge=1, le=365, description='Days until follow
    reasoning: str = Field(description='Clinical reasoning summary')

# The prompt that generates structured output
structured_prompt = f"""
Assess the patient and respond with ONLY a JSON object matching this schema:
{json.dumps(ClinicalAssessment.model_json_schema(), indent=2)}

Patient: Age 34, Female. Depression: 0.72, Anxiety: 0.65.
Symptoms: persistent low mood, insomnia, loss of appetite.
Therapy History: None.
"""

# Simulated model output (what the LLM would generate)
model_output = json.dumps({
    'severity': 'High',
    'confidence': 0.85,
    'risk_factors': [
        'Elevated depression score (0.72)',
        'Significant anxiety (0.65)',
        'Insomnia and appetite loss indicate somatic symptoms',
        'No prior therapy engagement'
    ],
    'protective_factors': [
        'Seeking assessment (engaged with care)',
        'Relatively young age (34)'
    ],
    'recommended_actions': [
        'Conduct PHQ-9 and GAD-7 screening',
        'Initiate CBT referral',
        'Consider SSRI (sertraline 50mg) if CBT insufficient after 4 weeks',
        'Sleep hygiene psychoeducation'
    ],
```

```
    'follow_up_days': 14,
    'reasoning': 'Co-occurring depression and anxiety with somatic symptoms
                 'indicate moderate-to-severe presentation. No prior treatme
                 'history suggests this is a first episode. Early interventi
                 'with evidence-based therapy is critical.'
})

# Parse and validate with Pydantic
assessment = ClinicalAssessment.model_validate_json(model_output)

print('=== Validated Clinical Assessment ===')
print(f'Severity: {assessment.severity.value}')
print(f'Confidence: {assessment.confidence:.0%}')
print(f'Risk Factors: {len(assessment.risk_factors)}')
for rf in assessment.risk_factors:
    print(f'  - {rf}')
print(f'Recommendations: {len(assessment.recommended_actions)}')
for ra in assessment.recommended_actions:
    print(f'  - {ra}')
print(f'Follow-up: {assessment.follow_up_days} days')

print(f'\n💡 Pydantic validates types, ranges, and required fields automatic
print(f'   If the model outputs invalid JSON, you catch it before it reaches
```

4. LLM-as-Judge Evaluation

Problem: How do you evaluate the quality of free-form text generation at scale?

Solution: Use a strong LLM to evaluate a weaker model's output.

Approaches

Method	How	Best For
Rubric scoring	Score 1-5 on specific criteria	Detailed quality analysis
Pairwise comparison	Which of two outputs is better?	Model comparison
Reference-based	How close to a gold answer?	When references exist

Known Biases

- **Position bias**: Prefers the first option in pairwise comparison
- **Verbosity bias**: Prefers longer, more detailed answers
- **Self-preference**: Models prefer their own outputs

In [ ]:
```
# ─── LLM-as-Judge rubric for clinical responses ──────────────────

evaluation_rubric = {
    'Clinical Accuracy': {
        5: 'All clinical facts are correct and evidence-based',
        3: 'Mostly accurate with minor omissions',
```

```
        1: 'Contains clinically inaccurate or dangerous information',
    },
    'Completeness': {
        5: 'Addresses all relevant aspects (risk factors, protective factors
        3: 'Addresses most aspects but misses some',
        1: 'Major gaps in the assessment',
    },
    'Actionability': {
        5: 'Clear, specific next steps that a clinician can follow',
        3: 'Some actionable recommendations but vague',
        1: 'No clear action items',
    },
    'Safety': {
        5: 'Appropriately identifies high-risk situations and recommends urg
        3: 'Identifies most risks but may understate severity',
        1: 'Misses critical safety concerns',
    },
}

def build_judge_prompt(response_to_evaluate, patient_info, rubric):
    """Build a prompt for LLM-as-judge evaluation."""
    rubric_text = ''
    for criterion, levels in rubric.items():
        rubric_text += f'\n{criterion}:\n'
        for score, description in sorted(levels.items(), reverse=True):
            rubric_text += f'  {score}: {description}\n'

    return (
        f'You are an expert clinical psychologist evaluating an AI-generated
        f'mental health assessment. Rate the response on each criterion (1-5
        f'### Patient Information:\n{patient_info}\n\n'
        f'### AI Response to Evaluate:\n{response_to_evaluate}\n\n'
        f'### Evaluation Rubric:\n{rubric_text}\n'
        f'### Your Evaluation (JSON):\n'
        f'Respond with a JSON object: {{"Clinical Accuracy": <1-5>, '
        f'"Completeness": <1-5>, "Actionability": <1-5>, "Safety": <1-5>, '
        f'"reasoning": "<brief explanation>"}}'
    )

# Simulated evaluation scores for 3 different model responses
model_scores = {
    'Model A (base)':     {'Clinical Accuracy': 3, 'Completeness': 2, 'Actio
    'Model B (fine-tuned)': {'Clinical Accuracy': 4, 'Completeness': 4, 'Act
    'Model C (RAG+FT)':   {'Clinical Accuracy': 5, 'Completeness': 5, 'Actio
}

# Visualise comparison
criteria = list(evaluation_rubric.keys())
fig, ax = plt.subplots(figsize=(10, 6))
x = np.arange(len(criteria))
width = 0.25

for i, (model_name, scores) in enumerate(model_scores.items()):
    values = [scores[c] for c in criteria]
    ax.bar(x + i * width, values, width, label=model_name)
```

```
ax.set_xticks(x + width)
ax.set_xticklabels(criteria, rotation=15)
ax.set_ylabel('Score (1-5)')
ax.set_title('LLM-as-Judge: Model Comparison on Clinical Assessment Quality'
ax.set_ylim(0, 5.5)
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

print('💡 RAG + fine-tuning outperforms either approach alone.')
print('   LLM-as-judge scales evaluation to thousands of examples.')
```

5. Function Calling & Tool Use

LLMs can't do everything — they can't query databases, call APIs, or perform calculations accurately.

Function calling lets the model decide *which tool to use* and *with what arguments*.

```
User: "What's the PHQ-9 score interpretation for 17?"
  ↓
Model decides: call severity_lookup(score=17, scale="PHQ-9")
  ↓
Tool returns: {"interpretation": "Moderately Severe",
"range": "15-19"}
  ↓
Model responds: "A PHQ-9 score of 17 indicates moderately
severe depression (range 15-19)."
```

In [ ]:
```
# ── Define tools for a mental health system ──────────────────────

# Tool definitions (JSON Schema format — same as OpenAI function calling)
tool_definitions = [
    {
        'name': 'severity_lookup',
        'description': 'Look up severity interpretation for a clinical score
        'parameters': {
            'type': 'object',
            'properties': {
                'score': {'type': 'number', 'description': 'The raw score'},
                'scale': {'type': 'string', 'enum': ['PHQ-9', 'GAD-7', 'C-SS
            },
            'required': ['score', 'scale']
        }
    },
    {
        'name': 'treatment_guidelines',
        'description': 'Retrieve evidence-based treatment guidelines for a c
        'parameters': {
            'type': 'object',
            'properties': {
                'condition': {'type': 'string', 'description': 'The conditic
```

```
                'severity': {'type': 'string', 'enum': ['mild', 'moderate',
            },
            'required': ['condition', 'severity']
        }
    },
    {
        'name': 'schedule_appointment',
        'description': 'Schedule a follow-up appointment for a patient',
        'parameters': {
            'type': 'object',
            'properties': {
                'patient_id': {'type': 'string'},
                'urgency': {'type': 'string', 'enum': ['routine', 'urgent',
                'days_from_now': {'type': 'integer', 'minimum': 0, 'maximum'
            },
            'required': ['patient_id', 'urgency', 'days_from_now']
        }
    }
]

# Tool implementations
def severity_lookup(score, scale):
    """Look up score interpretation."""
    ranges = {
        'PHQ-9': [(0, 4, 'Minimal'), (5, 9, 'Mild'), (10, 14, 'Moderate'),
                  (15, 19, 'Moderately Severe'), (20, 27, 'Severe')],
        'GAD-7': [(0, 4, 'Minimal'), (5, 9, 'Mild'), (10, 14, 'Moderate'),
                  (15, 21, 'Severe')],
    }
    for low, high, label in ranges.get(scale, []):
        if low <= score <= high:
            return {'interpretation': label, 'range': f'{low}-{high}', 'scor
    return {'interpretation': 'Unknown', 'score': score}

def treatment_guidelines(condition, severity):
    """Return evidence-based guidelines."""
    guidelines = {
        ('depression', 'mild'): 'Watchful waiting, exercise prescription, se
        ('depression', 'moderate'): 'CBT and/or SSRI (sertraline 50mg or flu
        ('depression', 'severe'): 'Combined SSRI + CBT, consider psychiatric
        ('anxiety', 'mild'): 'Self-help resources, relaxation techniques',
        ('anxiety', 'moderate'): 'CBT (individual or group) and/or SSRI',
        ('anxiety', 'severe'): 'SSRI/SNRI + CBT, avoid benzodiazepines long-
    }
    key = (condition.lower(), severity.lower())
    return {'condition': condition, 'severity': severity,
            'guideline': guidelines.get(key, 'No specific guideline found')}

# Execute a tool call
tool_registry = {
    'severity_lookup': severity_lookup,
    'treatment_guidelines': treatment_guidelines,
}

# Simulated model tool calls
tool_calls = [
```

```
    {'name': 'severity_lookup', 'arguments': {'score': 17, 'scale': 'PHQ-9'}
    {'name': 'treatment_guidelines', 'arguments': {'condition': 'depression'
]

print('=== Executing Tool Calls ===')
for call in tool_calls:
    fn = tool_registry[call['name']]
    result = fn(**call['arguments'])
    print(f'\nTool: {call["name"]}({call["arguments"]})')
    print(f'Result: {json.dumps(result, indent=2)}')

print('\n💡 The model decides WHICH tool to call and WITH WHAT arguments.')
print('   Your code executes the tool and feeds results back to the model.')
```

6. Agents & Multi-Step Reasoning

An **agent** is an LLM that can:

1. **Observe** the current state
2. **Think** about what to do next
3. **Act** by calling a tool
4. **Repeat** until the task is complete

ReAct Pattern (Reason + Act)

```
Thought: I need to assess this patient's depression severity.
Action: severity_lookup(score=17, scale='PHQ-9')
Observation: {"interpretation": "Moderately Severe", "range":
"15-19"}

Thought: The score is moderately severe. I should look up
treatment guidelines.
Action: treatment_guidelines(condition='depression',
severity='severe')
Observation: {"guideline": "Combined SSRI + CBT, consider
psychiatric referral"}

Thought: I now have enough information to provide a
recommendation.
Action: FINAL_ANSWER
```

In [ ]:
```
# ── Simple ReAct agent for mental health triage ──────────────────

class MentalHealthAgent:
    """A simple ReAct-style agent for clinical triage."""

    def __init__(self, tools):
        self.tools = tools
        self.trace = []  # Record of thoughts, actions, observations

    def think(self, observation):
```

```
        """Determine next action based on observation.
        In production, this would be an LLM call.
        Here we simulate with rule-based logic."""
        self.trace.append({'type': 'observation', 'content': str(observation
        return observation  # In prod: LLM generates the thought

    def act(self, tool_name, **kwargs):
        """Execute a tool and record the result."""
        self.trace.append({'type': 'action', 'tool': tool_name, 'args': kwar
        result = self.tools[tool_name](**kwargs)
        self.trace.append({'type': 'result', 'content': result})
        return result

    def run_triage(self, patient_data):
        """Run a complete triage workflow."""
        print('=== Agent Triage Workflow ===')

        # Step 1: Assess depression severity
        print('\nThought: First, I need to assess the depression severity.')
        dep_result = self.act('severity_lookup',
                              score=patient_data['depression_score'],
                              scale='PHQ-9')
        print(f'Action: severity_lookup(score={patient_data["depression_scor
        print(f'Observation: {dep_result}')

        # Step 2: Get treatment guidelines based on severity
        severity_map = {'Minimal': 'mild', 'Mild': 'mild', 'Moderate': 'mode
                        'Moderately Severe': 'severe', 'Severe': 'severe'}
        severity = severity_map.get(dep_result['interpretation'], 'moderate'

        print(f'\nThought: Depression is {dep_result["interpretation"]}. '
              f'Let me get treatment guidelines for {severity} depression.')
        tx_result = self.act('treatment_guidelines',
                             condition='depression', severity=severity)
        print(f'Action: treatment_guidelines(condition="depression", severit
        print(f'Observation: {tx_result}')

        # Step 3: Final recommendation
        print(f'\nThought: I have enough information. Generating final recom
        recommendation = {
            'patient': patient_data,
            'assessment': dep_result['interpretation'],
            'treatment': tx_result['guideline'],
            'follow_up': '7 days' if severity == 'severe' else '14 days'
        }
        print(f'\n=== Final Recommendation ===')
        for k, v in recommendation.items():
            print(f'  {k}: {v}')

        return recommendation

# Run the agent
agent = MentalHealthAgent(tools=tool_registry)
result = agent.run_triage({
    'name': 'Patient #1247',
    'depression_score': 17,
```

```
    'anxiety_score': 12,
    'age': 34
})

print(f'\n💡 The agent broke down the triage into 3 systematic steps.')
print(f'   Each step uses a specific tool — no hallucination of clinical dat
```

7. Cost & Latency Optimization

Production LLM applications must balance **quality**, **cost**, and **speed**.

Strategy	Cost Reduction	Latency Reduction	Quality Impact
Prompt caching	50-90% on repeated queries	~0 (cache hit)	None
Shorter prompts	~Linear with tokens	Linear	Minor
Batching	API overhead reduction	Higher throughput	None
Model routing	60-80% (small model for easy tasks)	Faster for easy tasks	Minimal if done well
Smaller model	5-20x cheaper	2-5x faster	Moderate
Fine-tuning	Shorter prompts needed	Same	Often better

In [ ]:
```
# ─── Model routing: complexity-based dispatch ──────────────────────

def estimate_complexity(query):
    """Estimate query complexity for model routing."""
    complexity_signals = {
        'multi_step': bool(re.search(r'(and|also|additionally|then)', query.
        'reasoning': bool(re.search(r'(why|how|explain|compare|analyse)', qu
        'clinical_risk': bool(re.search(r'(suicid|self.harm|crisis|emergency
        'long_query': len(query.split()) > 30,
    }
    score = sum(complexity_signals.values())
    return 'complex' if score >= 2 else 'simple'

def route_to_model(query):
    """Route query to appropriate model based on complexity."""
    complexity = estimate_complexity(query)

    models = {
        'simple': {'name': 'Phi-3-mini (3.8B)', 'cost_per_1k': 0.0001, 'late
        'complex': {'name': 'GPT-4 (1.8T est.)', 'cost_per_1k': 0.03, 'later
    }
    return models[complexity], complexity

# Test routing
test_queries = [
    'What is the PHQ-9 cutoff for moderate depression?',
```

```
    'Explain why this patient with suicidal ideation and treatment-resistant
    'List SSRI side effects.',
    'How should I manage a crisis situation involving self-harm and also ass
]

print('=== Model Routing Results ===')
total_cost_routed = 0
total_cost_always_large = 0

for query in test_queries:
    model, complexity = route_to_model(query)
    total_cost_routed += model['cost_per_1k']
    total_cost_always_large += 0.03  # always using GPT-4
    print(f'\n  [{complexity:7s}] → {model["name"]:20s} (${model["cost_per_1
    print(f'           "{query[:60]}..."')

savings = (1 - total_cost_routed / total_cost_always_large) * 100
print(f'\n=== Cost Comparison ===')
print(f'  Always GPT-4:  ${total_cost_always_large:.4f} per 4 queries')
print(f'  With routing:  ${total_cost_routed:.4f} per 4 queries')
print(f'  Savings:       {savings:.0f}%')

print(f'\n💡 Model routing sends easy queries to cheap/fast models.')
print(f'   Only complex or high-stakes queries go to the expensive model.')
```

In [ ]:

```
# ── Token cost estimation ──────────────────────────────────────

def estimate_tokens(text):
    """Rough token estimation (~4 chars per token for English)."""
    return len(text) // 4

def estimate_cost(prompt, completion, model='gpt-4'):
    """Estimate API cost for a single request."""
    pricing = {
        'gpt-4':       {'input': 0.03, 'output': 0.06},   # per 1K tokens
        'gpt-3.5':     {'input': 0.001, 'output': 0.002},
        'phi-3-mini':  {'input': 0.0001, 'output': 0.0001},  # self-hosted e
    }

    input_tokens = estimate_tokens(prompt)
    output_tokens = estimate_tokens(completion)
    rates = pricing[model]

    cost = (input_tokens * rates['input'] + output_tokens * rates['output'])
    return {'input_tokens': input_tokens, 'output_tokens': output_tokens,
            'cost': cost, 'model': model}

# Compare costs for different prompting strategies
strategies = {
    'Zero-shot': {'prompt_len': 200, 'output_len': 100},
    'Few-shot (3 examples)': {'prompt_len': 800, 'output_len': 150},
    'CoT': {'prompt_len': 300, 'output_len': 400},
    'Self-consistency (5x CoT)': {'prompt_len': 1500, 'output_len': 2000},
}

print('=== Cost Comparison per Request (GPT-4 pricing) ===')
```

```
costs = []
for strategy, params in strategies.items():
    prompt = 'x' * params['prompt_len'] * 4
    completion = 'x' * params['output_len'] * 4
    result = estimate_cost(prompt, completion, model='gpt-4')
    costs.append(result['cost'])
    print(f'  {strategy:30s}: ~{result["input_tokens"]:4d} in + {result["out

# Visualise
fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.bar(strategies.keys(), costs, color=['#3498db', '#e67e22', '#2ecc7
ax.set_ylabel('Cost per request ($)')
ax.set_title('Prompting Strategy Cost Comparison (GPT-4)')
for bar, cost in zip(bars, costs):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.001,
            f'${cost:.4f}', ha='center', fontsize=10)
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()

print('💡 Self-consistency costs 5x more but catches reasoning errors.')
print('   Use it for high-stakes decisions (risk assessment), not routine qu
```

🧪 Exercises

1. **Few-shot classification**: Write a few-shot prompt with 3 examples that classifies patient messages into urgency levels (low/medium/high/critical). Test with 5 new messages.

2. **Self-consistency**: Run the same CoT risk assessment prompt 5 times (simulate different temperature outputs). Implement a weighted majority vote where longer reasoning gets more weight.

3. **Structured output**: Create a Pydantic model for a structured therapy session summary with fields: session_date, patient_mood (1-10), topics_discussed (list), homework_assigned (list), next_session_date, therapist_notes. Parse simulated LLM output into it.

4. **Agent with tools**: Build a 2-tool agent that can (a) look up treatment guidelines and (b) assess severity scores. Have it handle: "Patient has a PHQ-9 of 22. What treatment do you recommend?"

✅ Key Takeaways

1. **Prompt engineering is systematic** — structure (system/user/assistant), examples, and formatting matter enormously
2. **Chain-of-thought** prompting dramatically improves reasoning accuracy by making the model show its work

3. **Structured output + Pydantic** makes LLM output production-ready with automatic validation
4. **LLM-as-judge** enables scalable evaluation but has known biases (position, verbosity, self-preference)
5. **Function calling** lets LLMs interact with external systems — no more hallucinated data
6. **Agents (ReAct)** extend LLMs from single-turn answers to multi-step problem solving
7. **Cost optimization** (caching, routing, batching) is essential — self-consistency costs 5x but is worth it for critical decisions

Next: 16 — Evaluation & Debugging — rigorously measuring model quality and debugging failures